

* Docker best practices >

Шпаргалка Dockerfile

FROM, COPY, ADD и другие...



Просто Devops



Docker Best Practices

БЛОК 1: База и подготовка

Правило первое.

Минимальный базовый образ (Alpine/Slim/Distroless/Scratch)

Alpine - 5MB, Slim ~50MB, Distroless/Scratch - без shell. Меньше размер = меньше уязвимостей.

× FROM ubuntu:latest # 200+ MB

✓ FROM scratch # 0 B (для статических бинарников)

Правило второе.

Фиксация версий. Правильные теги образов. (НИКАКИХ :latest)

Сегодня **latest** - это Node 18, а завтра - это Node 22. Надо указывать конкретные теги для воспроизводимости

× FROM python:latest

✓ FROM python:3.11.4-slim

Правило третье.

Файл .dockerignore

Исключать **.git, node_modules, .env, логи и подобное** из контекста сборки.
Ускоряет сборку, защищает секреты от попадания в образ.

```
# .dockerignore
.git
node_modules/
.env
```



Docker Best Practices

БЛОК 2: Оптимизация сборки

Правило четвертое.

Порядок слоёв и кэширование

Изменился слой → все следующие пересобираются.

Поэтому зависимости и всё, что меняется РЕДКО – вверху, а код – внизу.

`COPY package*.json ./` # сначала только манифесты

`RUN npm ci` # зависимости кэшируются

`COPY ..` # код меняется часто – поэтому в конце

Правило пятое.

Объединять RUN + чистить кэш в одном слое

Каждый RUN = новый слой. Удалённые файлы остаются в предыдущих слоях!

Объединяем через &&, переносим строки с помощью \:

`RUN apt-get update && apt-get install -y curl && \` # всё в одном RUN

`rm -rf /var/lib/apt/lists/*` # чистим в том же слое

Правило шестое.

COPY вместо ADD

ADD магически распаковывает архивы и качает URL.

COPY – это простое копирование, оно предсказуемо.

× `ADD https://prostodevops.ru/file.tar.gz /app/`

✓ `COPY local.txt /app/` # или `RUN curl + tar`



Docker Best Practices

БЛОК 2: Оптимизация сборки

Правило седьмое (очень важное).

Multi-stage build

Для уменьшения размера конечного образа надо использовать multi-stage.

Собираем в тяжёлом образе (условно golang, который весит примерно 1GB), а запускаем в **лёгком** (alpine, который всего 5MB).

```
FROM golang:1.21 AS builder # этап 1: сборка
```

```
RUN go build -o /app
```

```
FROM alpine # этап 2: чистый образ
```

```
COPY --from=builder /app /app # копируем ТОЛЬКО бинарник из builder
```

Правило восьмое.

Использовать BuildKit

BuildKit – это движок для сборки. В новых версиях Docker он используется по умолчанию.

Старый билдер тупо идёт сверху вниз. BuildKit видит, что можно параллелить, и собирает быстрее.

Однако если версия старая, делаем:

```
export DOCKER_BUILDKIT=1 # включаем (в новых Docker уже по умолчанию)
```

```
docker build .
```



Docker Best Practices

БЛОК 3: Безопасность

Правило девятое.

*Не работать под **root***

По дефолту **root** в контейнере = **root** на хосте при побеге из контейнера.

Поэтому создаём юзера и переключаемся на него:

```
RUN addgroup -S app && adduser -S app -G app # создаём юзера
COPY --chown=app:app . . # файлы сразу его
USER app # переключаемся
```

Правило десятое.

Build Secrets (HE ARG/ENV!)

ARG записывается в **docker history** навсегда.

Поэтому для секретов используем **--mount=type=secret** (кстати для этого нужен BuildKit).

× **ARG GITHUB_TOKEN=ghp_xxx**

✓ **docker build --secret id=token,src=./token.txt**

В Dockerfile:

```
RUN --mount=type=secret,id=token \ # монтируем секрет
    TOKEN=$(cat /run/secrets/token) && npm i # используем и всё
```



Docker Best Practices

БЛОК 4: Стабильность

Правило одиннадцатое.

PID 1 и Graceful Shutdown

Bash **не** передаёт сигналы детям!

Kubernetes шлёт SIGTERM → Bash молчит → SIGKILL через 30 сек.

Поэтому используем `exec` или `Tini`.

`exec node app.js` # в `entrypoint.sh` - заменяет `bash` на `app`

или в `Dockerfile`:

`ENTRYPOINT ["/sbin/tini", "--"]` # `tini` проксирует сигналы

Правило двенадцатое.

Использовать HEALTHCHECK'и

Docker считает контейнер живым, если процесс **есть**. Поэтому может быть ситуация, когда **процесс завис**, но **докеру норм**, так как **контейнер жив**.

Добавляем `HEALTHCHECK` в `Dockerfile`:

`HEALTHCHECK --interval=30s --timeout=3s \` # каждые 30с, таймаут 3с

`CMD curl -f http://localhost:8080/health || exit 1` # `exit 1` = unhealthy

Правило тринадцатое.

Линтер Hadolint

Статический анализ `Dockerfile`. Ловит `latest`, `sudo`, отсутствие очистки кэша.

`hadolint Dockerfile` # локально

`docker run --rm -i hadolint/hadolint < Dockerfile` # в CI

